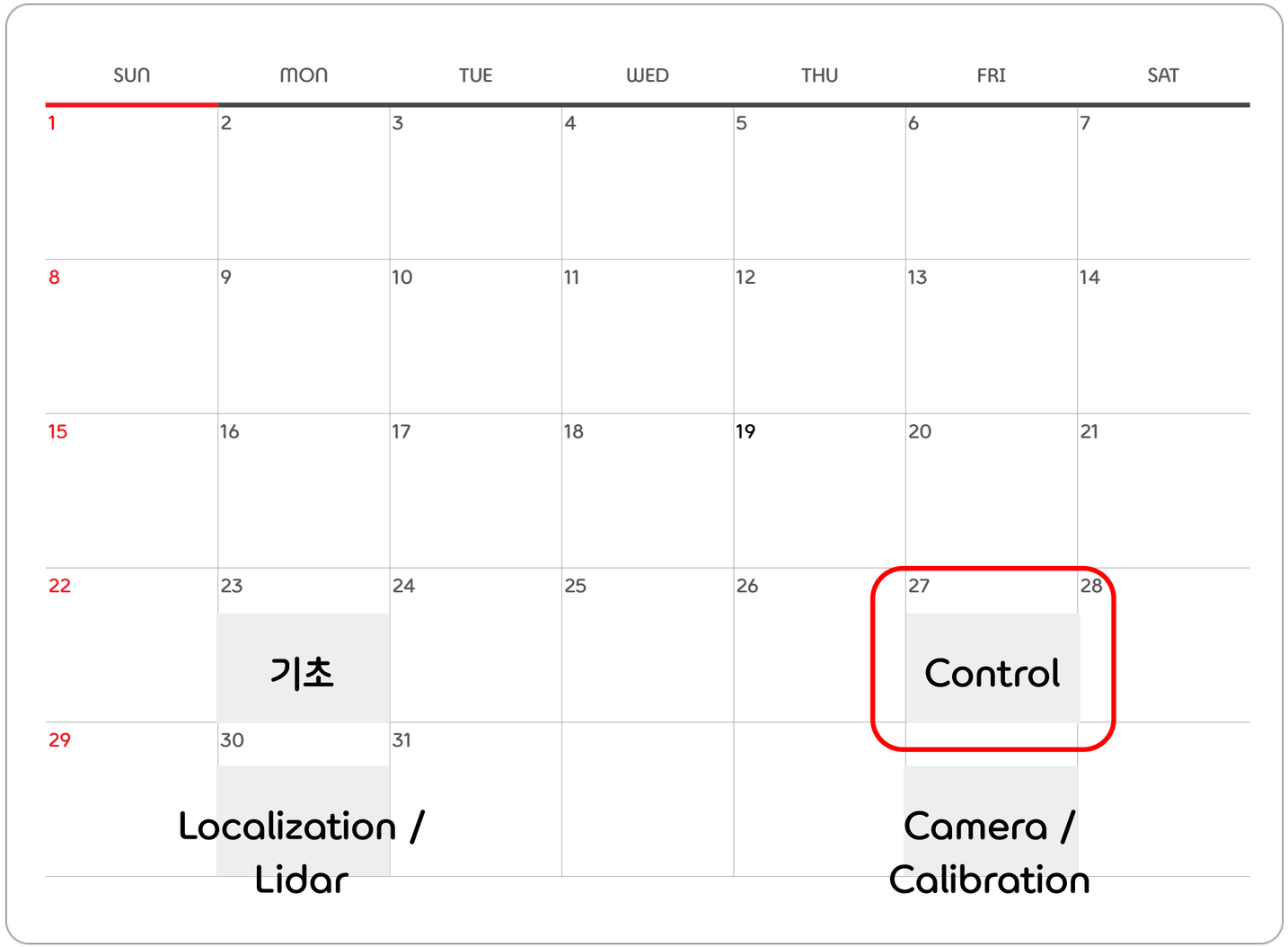
Clothoid-R

# Control

2026.03

# 세미나 일정

1. 기초 ROS / Gazebo
2. Control
3. Localization / Lidar
4. Camera / Calibration
5. Planning
6. 통합 자율주행




세미나 교육을 듣고 자기가 제일 재밌어 보이고 하고 싶은 파트 선택하면 됨

# 맵 따보기 (실습)

Using GPS

1. 가제보를 실행시킨다.

gz sim -r ~/Autonomous-Racing-Simulator/simulate_ws/src/server/map/racemap.sdf

2. 차량을 스폰시킨다.

ros2 launch server spawn_car.launch.py

3. 차량을 움직일 수 있게 파이썬 코드를 실행시킨다.

python3 src/server/src/key_teleop.py

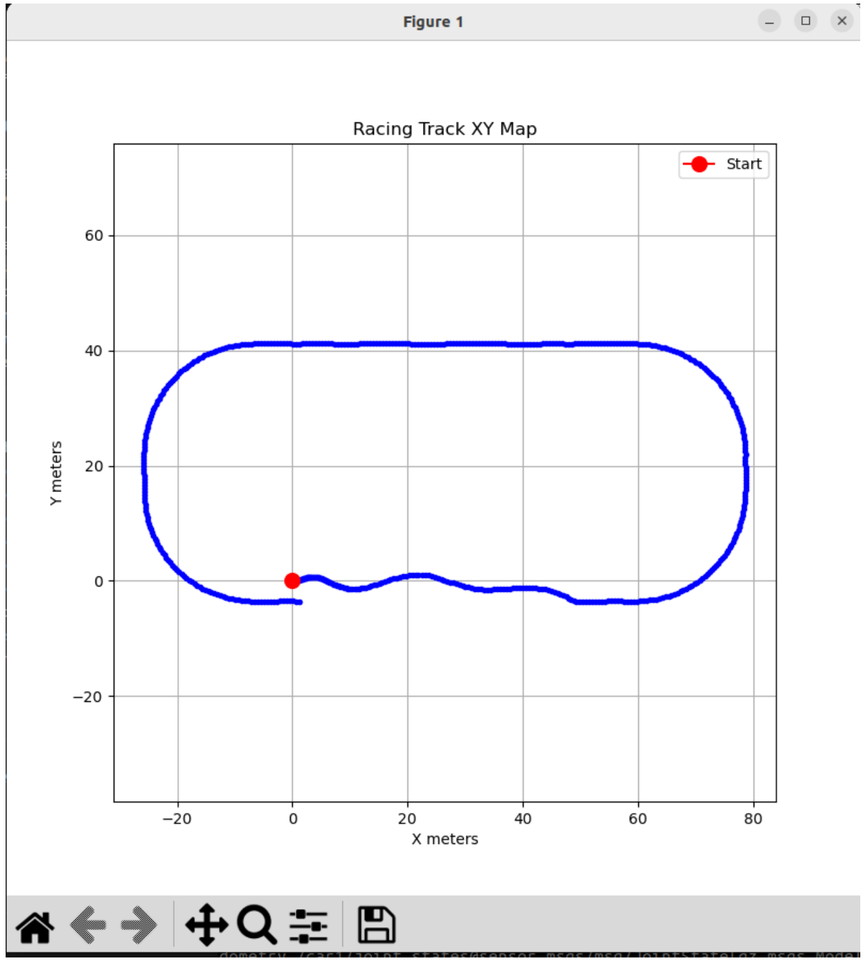
4. 차량에 설치된 GPS를 이용하여 맵의 경로를 추출한다.

python3 src/path_maker/path_maker.py

https://docs.ros.org/en/kilted/Tutorials/Beginner-CLI-Tools/Configuring-ROS2-Environment.html

# 맵 따보기 (결과)

Using GPS




https://docs.ros.org/en/kilted/Tutorials/Beginner-CLI-Tools/Configuring-ROS2-Environment.html

# Control

제어에 대하여

## 1 제어란?

기계, 장치, 시스템 또는 특정 상황이 원하는 목표 방향으로 작동하도록 통제하고 조절하는 것을 의미

## 2 제어의 궁극적인 목표

현재의 값과 목표값의 오차를 0으로 만드는 것

## 3 자율주행에서 제어로는 무엇이 있을까?

종방향제어(속도) - PID control
횡방향제어(조향) - Pure Pursuit, Stanley method, MPC
트랜드 : E2E(학습을 통해 인지하고 판단하고 제어를 동시에 함)

# Control

## Open Loop, Closed Loop

### Open loop control

장점
1. 시스템을 설계하는데 있어 복잡하지 않다.

단점
1. 외부조건(외란)의 변화에 대처할 수 없다.
2. 목표값과 오차가 많이 발생할 수 있다.

### Closed loop control

장점
1. 외부조건의 변화에 대처할 수 있다.
2. 제어계의 특성을 향상시킬 수 있다.
3. 목표값에 정확히 도달할 수 있다.

단점
1. 복잡해지고 값이 비싸진다.
2. 제어계 전체가 불안정해질 수 있다.

### Open Loop Control (개루프 제어)

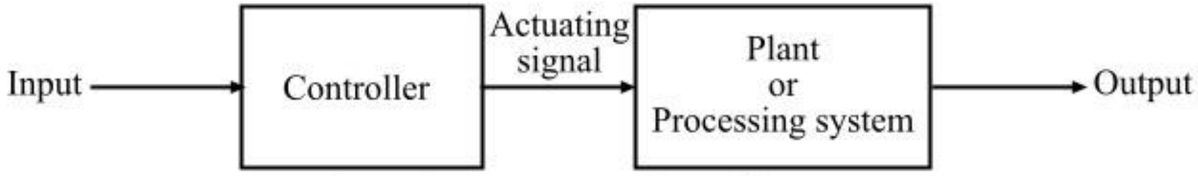



Figure 1 - Open Loop Control System

### Closed Loop Control (폐루프 제어)

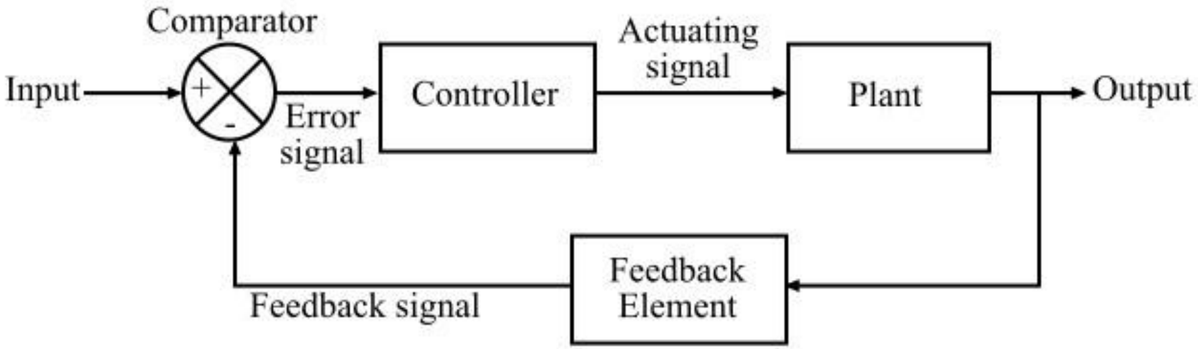
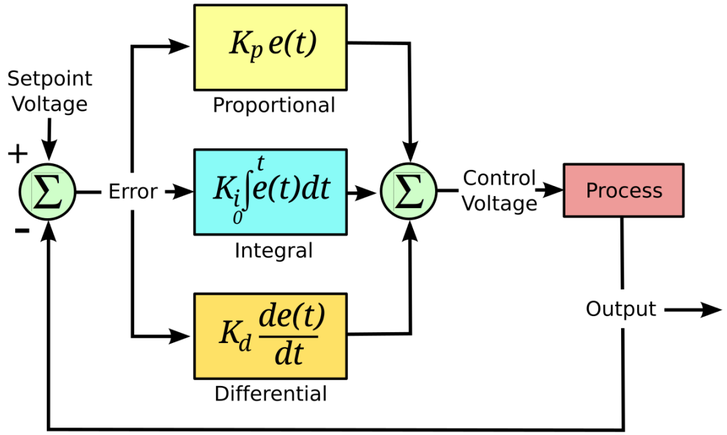



Figure 2 - Closed Loop Control System

그림 출처 : https://www.tutorialspoint.com/article/difference-between-open-loop-and-closed-loop-control-system

# 종방향제어

PID control




•비례(Proportional) 제어:
현재 오차 에 비례하여 제어 신호를 생성한다. 비례 이득이 크면 응답 속도가 빨라지고 오차에 민감하게 반응하지만, 과도응답에서 오버슈트가 증가할 수 있다.

•적분(Integral) 제어:
과거의 오차를 누적하여 제어한다. 적분 동작은 정상 상태에 남는 잔류 오차(steady-state error)를 제거하여 목표값에 수렴하도록 하지만, 누적된 오차로 인해 과도응답이 진동하거나 과도 시간이 길어질 수 있다.

•미분(Derivative) 제어:
오차의 변화율()을 예측하여 제어한다. 미분 동작은 빠르게 변하는 오차에 선제적으로 반응하여 과도응답의 오버슈트를 억제하고 시스템의 안정성을 높여준다.

**실습 해보기!**

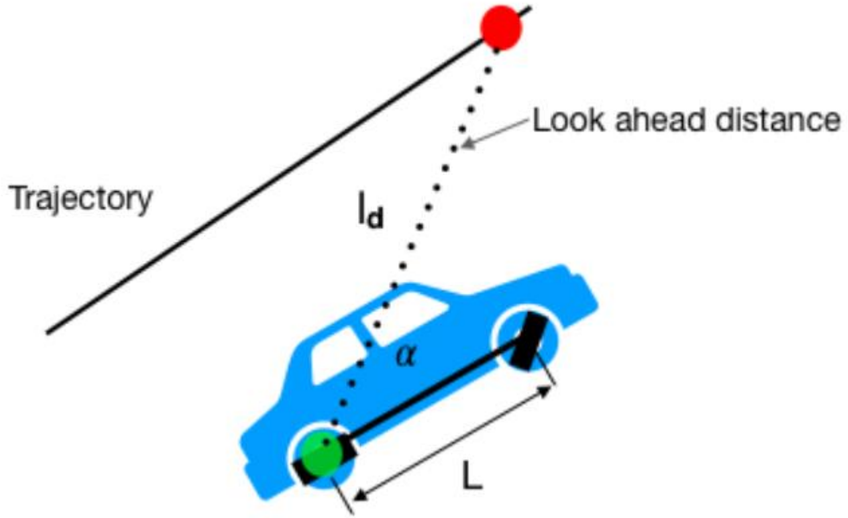
https://www.luisllamas.es/en/pid-controller-simulator/

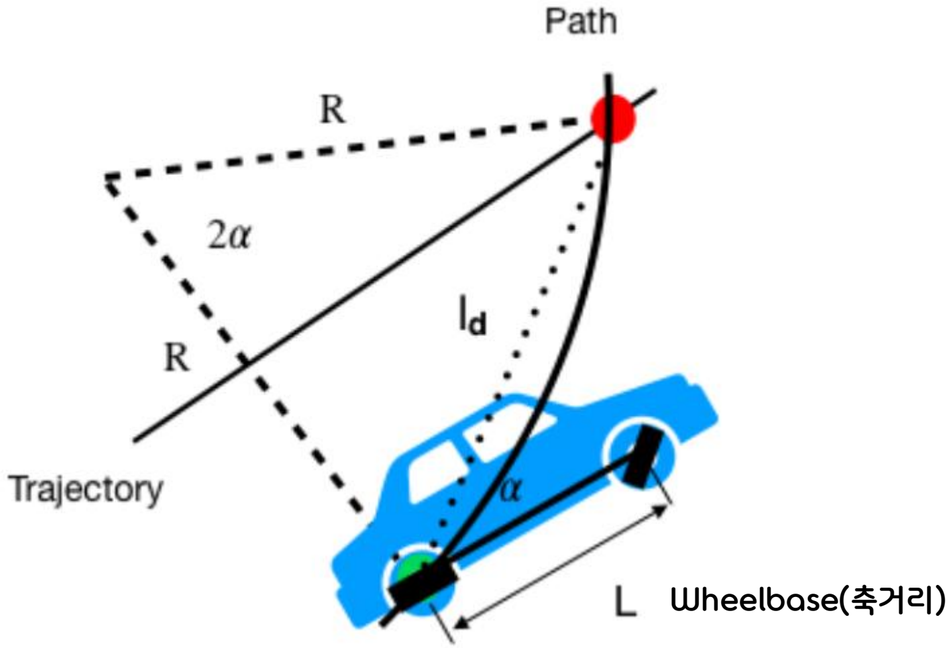
그림 출처 : https://commons.wikimedia.org/wiki/File:PID_loop.svg

# 횡방향제어

Pure Pursuit

Kinematic bicycle model은 단순화 된 자동차의 수학적 모델로
일반적인 주행 조건에서 차량 움직임을 매우 잘 포착하는 모델




출처 : https://koreatech-kroad.github.io/Pure-Pursuit-&-Stanely

# 제어 (실습)

## PID control, Pure Pursuit

### 1. 가제보를 실행시킨다.

gz sim -r ~/Autonomous-Racing-Simulator/simulate_ws/src/server/map/racemap.sdf

### 2. 차량을 스폰시킨다.

ros2 launch server spawn_car.launch.py

### 3. Path를 발행

ros2 run path_maker pub_path.py

### 4. PID 속도 제어, Pure Pursuit, Supervisor 런치 파일 실행

Ros2 launch controller all_controller.launch.py

# QnA

이상으로 세미나를 마칩니다.
궁금한 점은 질문해주세요.